# 5. Maszynowe wspomaganie obliczeń numerycznych i symbolicznych

## System komputerowy jako maszyna licząca

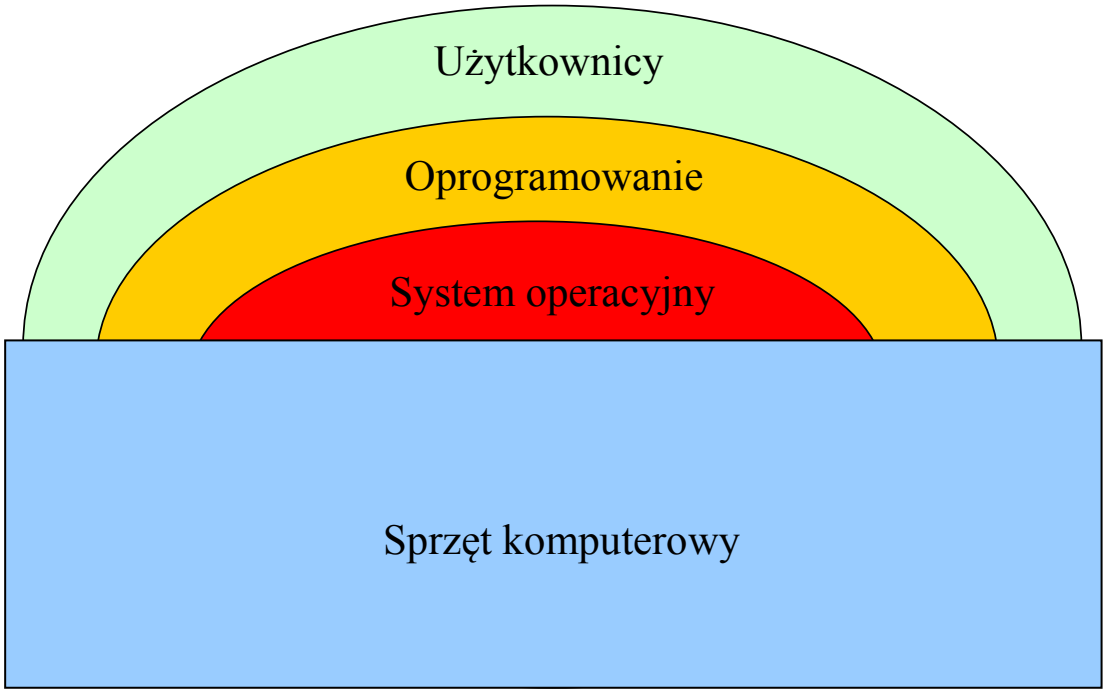
W odróżnieniu od większości pozostałych rodzajów maszyn liczących współczesny komputer cechuje się możliwością automatycznego, wielokrotnie powtarzalnego wykonywania dowolnego ciągu instrukcji zwanego programem, którego celem jest przetwarzanie dowolnej informacji. Pod tym względem komputery należą do grupy uniwersalnych, programowalnych maszyn o ogólnym zastosowaniu.

Od samego początku swego istnienia, tj. od powstania maszyny ENIAC w 1946 r., komputery stosuje się jako maszyny liczące do rozwiązywania problemów matematycznych głównie na drodze obliczeń numerycznych. Na przykład *ENIAC* (ang. *Electronic Numerical Integrator And Computer*) był wykorzystywany początkowo do obliczania trajektorii balistycznych, natomiast później między innymi do obliczeń parametrów tuneli aerodynamicznych, prognozowania numerycznego pogody, wykonywania symulacji w oparciu o liczby losowe, badania promieniowania kosmicznego, czy też do obliczenia wartości liczby **pi** z dokładnością do tysiąca miejsc po przecinku.

Komputery mogą również wykonywać zadania nienumeryczne, takie jak przetwarzanie tekstu, rozpoznawanie obrazu, tworzenie grafiki i animacji, generowanie dźwięku, zarządzanie bazami danych, przesyłanie informacji w sieci, maszynowe uczenie się, wydobywanie wiedzy, podejmowanie decyzji itd. W przypadkach tych komputer na ogół nie przetwarza informacji w postaci liczb, lecz inne formy informacji, takie jak teksty, obrazy, grafika, dźwięki, sygnały itp. Ponieważ jednakże komputer jest urządzeniem cyfrowym, więc dowolna informacja przetwarzana przez to urządzenie jest wewnętrznie reprezentowana w postaci ciągów liczb binarnych. Wobec tego informacja o formie nienumerycznej musi być wstępnie zamieniona na postać cyfrową, np.

poprzez odpowiednią digitalizację. Dotyczy to również informacji w postaci liczb zapisanych w systemie niebinarnym. A zatem nie zależnie od formy przetwarzanej informacji i charakteru związanego z tym zadania komputer zawsze wykonuje pewne obliczenia na liczbach, co uzasadnia umieszczenie tego urządzenia w rodzinie maszyn liczących.

Komputery jako maszyny liczące ze względu na ich złożoną budowę należy rozpatrywać w szerszym kontekście systemu komputerowego jako ogółu składników związanych bezpośrednio lub pośrednio z komputerem. Współczesne systemy komputerowe stanowią istotę technicznych środków informacji, a więc środków jej gromadzenia, przechowywania, przekształcania oraz przesyłania.




Przez system komputerowy rozumiemy na ogół bardzo złożony układ zawierający składniki fizyczne w postaci sprzętu komputerowego oraz składniki programowe, czyli oprogramowanie. Sprzęt to przede wszystkim takie zasoby systemu, jak procesor CPU, pamięć oraz urządzenia peryferyjne. Oprogramowanie obejmuje system operacyjny oraz rozmaite programy zwane również aplikacjami systemu. Do systemu komputerowego należy włączyć także samych użytkowników systemu.

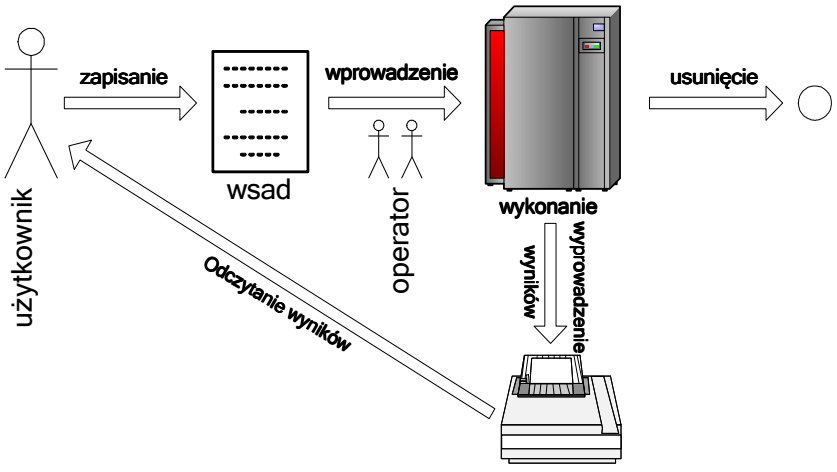
W początkowym okresie rozwoju systemów komputerowych nowe oprogramowanie składające się na system tworzone było od podstaw, a więc obok swej właściwej logiki (związanej z przeznaczeniem systemu) zawierało wiele funkcji przeznaczonych do zarządzania zasobami systemu. Oznaczało to, iż każdy program musiał sam sterować dostępem do pamięci, drukarki itp. Z czasem wszystkie wspólne, powtarzające się fragmenty oprogramowania wykonujące bezpośrednie operacje na sprzęcie wydzielono pod postacią specjalnego programu zwanego systemem operacyjnym.

Współczesne systemy operacyjne stanowią złożoną pośrednią warstwą programową pomiędzy sprzętem komputerowym a użytkownikiem, tj. człowiekiem lub innym systemem komputerowym. Zadaniem tej warstwy jest utworzenie wygodnego środowiska operacyjnego, w którym użytkownik może wykonać własne programy, czyli aplikacje systemu, zawierające kod związany z właściwym zastosowaniem całego systemu komputerowego. Wśród aplikacji można wyróżnić programy narzędziowe o specjalnym zastosowaniu (narzędzia administratora systemu, narzędzia programistyczne itp.) oraz zwykłe programy użytkowe (np. edytory tekstu, arkusze kalkulacyjne).

Podstawowy podział systemów komputerowych wykonuje się z punktu widzenia trybu wprowadzania zadań do systemu:

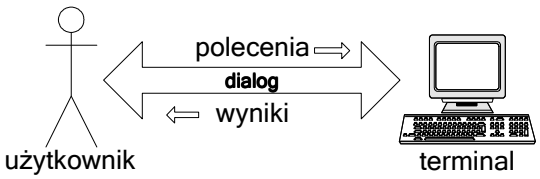
- **systemy wsadowe** - pierwsze proste systemy, które cechowały się brakiem możliwości bezpośredniego wprowadzania zadań do sprzętu przez użytkownika systemu.

Wszystkie czynności dot. danego zadania powierzane były operatorom systemu. Wsad, czyli zadanie do przetworzenia, składał się z kodu programu i wartości danych, a zapisywany był na specjalnym nośniku w postaci kart perforowanych. Praca systemu polegała na wprowadzeniu, wykonaniu i usunięciu jednego zadania. Wyniki obliczeń były wyprowadzane jedynie na proste urządzenia wyjściowe, np. drukarkę.



- **systemy konwersacyjne** – następna generacja systemów, zapewniających bezpośrednią kontrolę użytkownika systemu nad przetwarzanym zadaniem poprzez dialog z systemem.

Urządzeniem do wprowadzania zadań oraz wydawania poleceń sterujących ich przetwarzaniem jest w systemie tym terminal (monitor i klawiatura), który ponadto umożliwia podawanie danych wejściowych, przeglądanie końcowych wyników obliczeń, a także odbieranie bieżących odpowiedzi systemu.

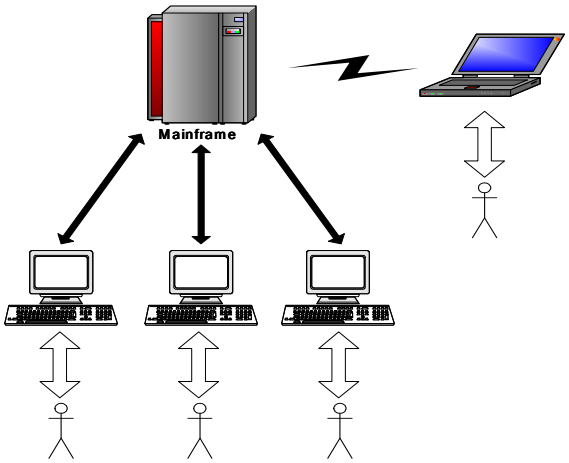



W systemach konwersacyjnych pojawiło się pojęcie interfejsu użytkownika jako warstwy dialogu pomiędzy użytkownikiem a systemem komputerowym, w której interakcja użytkownika z systemem składa się z wielu krótkich zdarzeń. Od momentu powstania koncepcji interfejsu użytkownika opracowano wiele różnych metod interakcji z użytkownikiem. Wygląd i funkcjonowanie interfejsu w dużym stopniu zależą od trybu wyświetlania informacji przez system na ekranie monitora. W trybie tekstowym najmniejszym elementem na ekranie podlegającym sterowaniu jest znak pewnego zbioru, najczęściej ASCII (ang. *American Standard Code for Information Interchange*), zaś w trybie graficznym – pojedynczy punkt geometryczny, czyli tzw. piksel (ang. *picture element*).

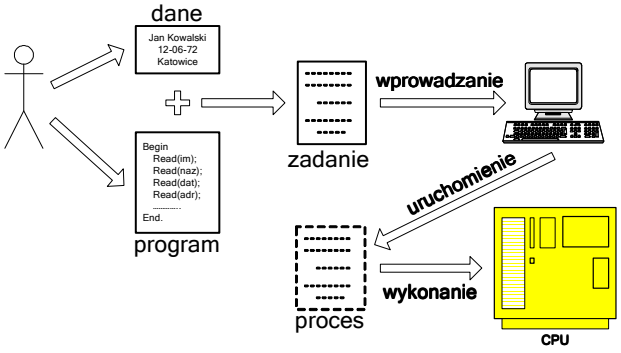
W interfejsie użytkownika wprowadzanie poleceń do systemu może odbywać się poprzez linię poleceń (DOS, UNIX) lub poprzez rozwijalne menu poleceń o strukturze hierarchicznej (Norton Commander, Midnight Commander). Takie rozwiązania są najczęściej obecne w systemach z wyświetlaniem tekstowych. Natomiast w systemach graficznych interakcja zachodzi poprzez graficzne obiekty, takie jak okienka, kontrolki, symbole zwane ikonami, czy menu poleceń (systemy MacOS, OS/2, Windows). Obsługa systemu polega zatem na manipulowaniu za pomocą kursora myszy obiektami graficznymi na ekranie, co wywołuje określone operacje wykonywane przez system. Taka metoda interakcji nosi nazwę graficznego interfejsu użytkownika, GUI (ang. *graphical user interface*).

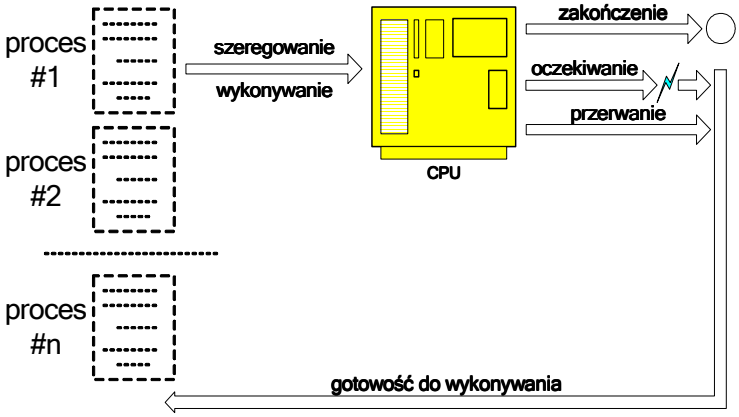
Podział systemów komputerowych wykonuje się również ze względu na dostęp użytkownika. Wyróżniamy zatem **systemy jednodostępne** oraz **wielodostępne**. Inny podział dotyczy trybu przetwarzania zadań i obejmuje **systemy jednozadaniowe** oraz **wielozadaniowe**.

Systemy wielodostępne pozwalają wielu użytkownikom korzystać z tego samego systemu komputerowego jednocześnie. Wielodostępność jest cechą charakterystyczną wielkich systemów komputerowych składających się z jednostki głównej (ang. *mainframe*) oraz licznych terminali. Komputery osobiste przeznaczone są zwykle dla jednego użytkownika. Jeżeli jednak komputer osobisty włączony jest do sieci lokalnej, w której może pełnić rolę serwera, to dostęp do jego zasobów również jest możliwy jednocześnie dla wielu użytkowników.



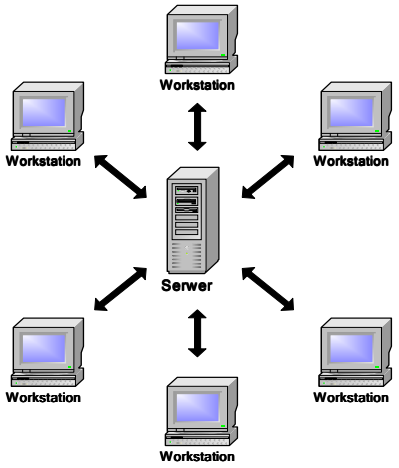

W systemie wielozadaniowym wiele zadań może być wykonywanych równocześnie i niezależnie od siebie. Wykonywane przez system zadanie na podstawie uruchomionego programu nazywamy procesem, stad systemy takie są również określane mianem systemów wieloprocesowych. W trakcie działania systemu istnieje kolejka procesów gotowych do wykonywania, spośród których tylko jeden jest zaszeregowany do wykonywania. Stan ten nie może trwać długo, stąd po upływie pewnego krótkiego czasu proces zostaje przerwany i powraca do kolejki. Proces może również przejść w stan oczekiwania na pewne zdarzenie, po którym wraca do kolejki, lub ulec zakończeniu. Procesor CPU wykonuje na przemian wiele różnych procesów dzięki przerwaniom i podziałowi czasu jego pracy. Przełączenia pomiędzy procesami występują tak często, że użytkownik systemu ma wrażenie jednoczesnego przetwarzania zadań; stanowi to istotę współbieżnego (pseudorównoległego) przetwarzania. Dzięki temu użytkownik może płynnie pracować z wieloma programami jednocześnie.



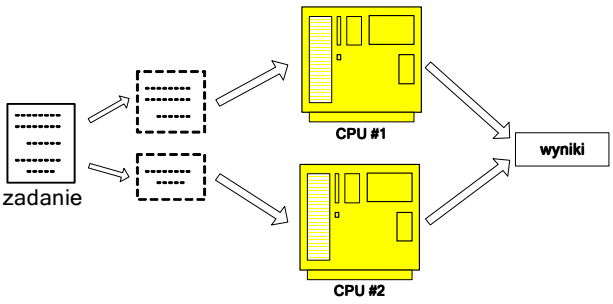



Zwiększenie szybkości wykonywanych zadań jest możliwe w systemach wieloprocesorowych zawierających więcej niż jeden procesor CPU lub procesor wielordzeniowy. W takim przypadku poszczególne procesy mogą być wykonywane równolegle przez osobne procesory (rdzenie). Systemy takie cechują się ścisła komunikacja pomiędzy procesorami poprzez pamięć dzieloną, zatem są to systemy ściśle powiązane (procesory dzielą zasoby).

W systemach rozproszonych rozdzielenie zadań zachodzi pomiędzy wiele luźno powiązanych procesorów, gdzie każdy procesor ma osobne zasoby sprzętowe (np. pamięć). Procesory komunikują się ze sobą poprzez różne łącza komunikacyjne i nierzadko same znacznie się różnią. Systemy rozproszone mogą posiadać dowolną strukturę organizacyjną, jednakże najczęściej spotyka się konfigurację typu klient-serwer, w której istnieje jeden komputer centralny będący serwerem systemu globalnego oraz wiele mniejszych komputerów klienckich z lokalnymi systemami typu *workstation.* Komputer serwera nierzadko posiada wiele procesorów (np. dwa, cztery), co wymaga stosowania dodatkowo systemu wieloprocesorowego.




Systemy wieloprocesorowe i/lub rozproszone często określamy mianem systemów równoległych ze względu na możliwość wykonywania przez kilka procesorów CPU systemu różnych fragmentów danego zadania w tym samym czasie. W przypadku obliczeń numerycznych równoległe wykonywanie zadań ma szczególne zastosowanie do rozwiązywania złożonych problemów dotyczących takich zagadnień, jak prognozowanie pogody, przewidywanie wstrząsów ziemi, wyznaczanie struktur białek, projektowanie lekarstw, otrzymywanie nowych materiałów.



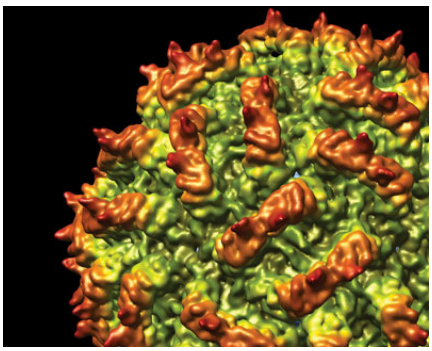

Obecnie największe systemy komputerowe, czyli tzw. superkomputery, wykonujące obliczenia równoległe najczęściej posiadają konfigurację prostego klastera bardzo wielu mniejszych komputerów połączonych w sieć o pewnej topologii (opis[1] przykładowych superkomputerów - patrz niżej). Algorytmy

[1] Cytaty i obrazy dotyczące przykładów superkomputerów pochodzą z encyklopedii Wikipedia (http://pl.wikipedia.org).

programu do wykonywania obliczeń numerycznych w trybie równoległym muszą przyjąć specjalną postać zależną od topologii klastera. Implementacja algorytmów obliczeń równoległych należy na ogół do bardzo trudnych przedsięwzięć programistycznych.



**Blue Gene** - projekt architektury komputerowej mający na celu stworzenie kilku następnych generacji superkomputerów. Projekt powstaje we współpracy firmy IBM z Lawrence Livermore National Laboratory oraz Departamentem Energii USA. Obecnie istnieją cztery projekty Blue Gene: BlueGene/L, BlueGene/C, BlueGene/P oraz BlueGene/Q. Pierwszy z nich, Blue Gene/L znajduje się na pierwszym miejscu listy 500 najszybszych komputerów na świecie. W/g benchmarku LINPACK dysponował on pierwotnie mocą obliczeniową równą 280,6 teraflopsom, która została zwiększona w 2007 roku do 478 teraflopsów ($10^{12}$ operacji zmiennoprzecinkowych na sekundę). Zaprezentowana przez IBM 26 czerwca 2007 druga generacja superkomputera, Blue Gene/P korzysta z 294 912 rdzeni procesora IBM PowerPC 450, z których każdy jest taktowany zegarem o częstotliwości 850 MHz. Cztery rdzenie tworzą jeden procesor. Superkomputer ten osiąga prędkość 1 petaflopsa ($10^{15}$ operacji zmiennoprzecinkowych na sekundę). Jest sto tysięcy razy szybszy niż przeciętny komputer biurowy. Procesory CPU montowane są na płytach, a każda płyta zawiera 32 jednostki CPU. Płyty zamocowane są w szafach. W jednej mieszczą się 32 płyty. Nowy system Blue Gene/P zawiera 72 takie szafy. Superkomputer można rozbudować do 216 szaf, czyli 884 736 rdzeni i wówczas osiągnie on wydajność 3 petaflopsów. Zgodnie z jego nazwą superkomputer ten pierwotnie stosowany był do symulacji struktury DNA i molekuł białka. Obecnie Blue Gene/L stosowany jest do wykonywania złożonych symulacji w różnych dziedzinach badań. (Fotografie obok przedstawiają jedną z szaf superkomputera BlueGene/L oraz strukturę 3D wirusa, patrz http://www.sdsc.edu/index.html)





**Earth Simulator** - dawniej największy, a obecnie siódmy co do wielkości (listopad 2005) superkomputer na świecie. Earth Simulator został skonstruowany przez firmę NEC. Znajduje się w Earth Simulator Centre koło Jokohamy w Japonii i działa od marca 2002. Jego budowa na pewien czas przerwała dominację USA w dziedzinie superkomputerów. Earth Simulator składa sie z 5120 procesorów i ma 10 terabajtów pamięci. Jego moc obliczeniowa wynosi 35.6 teraflopsów. W momencie budowy był 5 razy szybszy od drugiego największego w tym czasie superkomputera na świecie, IBM ASCI White ulokowanego w USA i używanego do modelowania wybuchów jądrowych. Earth Simulator jest używany do tworzenia wirtualnego modelu Ziemi. Może przeprowadzać obliczenia modelu opisującego całą atmosferę Ziemi z rozdzielczością 10 km. Dzięki tej symulacji można śledzić zmiany klimatyczne na naszym globie i przewidywać różnorodne zjawiska atmosferyczne. (Fotografie obok przedstawiają szafy superkomputera Earth Simulator oraz wyniki symulacji atmosfery Ziemi, patrz http://www.es.jamstec.go.jp/index.en.html)



# Złożoność programów obliczeniowych

Stopień złożoności kodu i tym samym użyteczność danego programu obliczeniowego można mierzyć na wiele sposobów. Najważniejszą miarą jest tzw. złożoność obliczeniowa algorytmów (ang. *computational complexity*) zaimplementowanych w programie. Miara ta odniesiona do czasu wyrażona jest przez liczbę wykonywanych w algorytmie elementarnych operacji, takich jak dodawanie, mnożenie lub porównywanie itp. Zbliżone do złożoności jest pojęcie efektywności algorytmu, związane z czasem działania kodu programu, który jest techniczną realizacją (implementacją) danego algorytmu.

Obok czasu w ocenie użyteczności programu można również poddać pamięć zajmowaną przez dane algorytmu lub inne zasoby systemu. Wobec tego wyróżniamy przynajmniej dwie główne miary złożoności obliczeniowej programu i jego algorytmów: złożoność czasową i przestrzenną. Badaniem algorytmów w zakresie wyznaczania ich charakterystyk wydajnościowych zajmuje się analiza algorytmów. W ramach analizy teoretycznej do wyznaczenia charakterystyki badanego algorytmu stosuje się narzędzia i metody matematyczne, wobec tego przedmiotem badań tej dziedziny może być dowolny algorytm w postaci abstrakcyjnej.

Ponieważ nie zawsze można otrzymać tą drogą zadawalające wyniki, więc w badaniach stosuje się również metody empiryczne. Niestety, analiza empiryczna jest zależna od konkretnej implementacji badanego algorytmu w postaci programu, języka programowania, translatora kodu źródłowego, a nawet od środowiska, w którym program jest wykonany.

Niezależnie od rodzaju analizy w badaniach danego algorytmu powinno uwzględniać się nie tylko jego treść, ale również postać danych wejściowych, jakie algorytm przekształca w wyniki. Aby rezultaty badań były jak najmniej zależne od rodzaju danych, na wejście algorytmu podaje się dane zarówno typowe, jak i wygenerowane losowo. W celu wyznaczenia maksymalnego czasu podaje się także dane złośliwe, dla których algorytm nie jest efektywny.

W praktyce programistycznej często stosuje się specjalne narzędzia, tzw. profilery, służące do badania charakterystyk wydajnościowych tworzonych programów, zwłaszcza zaimplementowanych algorytmów. Narzędzia te nie tylko wyświetlają statystyki wykonywania różnych fragmentów kodu, ale

również ułatwiają profilowanie kodu, czyli usuwanie instrukcji wpływających negatywnie na wydajność algorytmów.

Jednym z podstawowych czynników wpływający na czas wykonywania określonego algorytmu jest rozmiar (liczność) danych wejściowych, zwany w skrócie rozmiarem wejścia. Może to być rozmiar pliku do przeszukiwania, liczba wyrazów w sortowanym ciągu, liczba składników obliczanej sumy wyrażeń, np. stopień wielomianu, liczba kroków iteracyjnych w metodzie numerycznej, liczba wywołań funkcji rekurencyjnej, wymiar i rząd mnożonych macierzy itd.

Niech dany będzie algorytm $A$ oraz jego techniczna realizacja w postaci programu $P$. Przyjmujemy dalej następujące uproszczenie, iż niezależnie od charakteru danych wejściowych czas $T$ wykonywania badanego algorytmu $A$ jest jedynie funkcją liczności $n$ danych:

$$T_A = T_A(n) \quad n \geq 0$$

Czas działania implementującego algorytm programu jest wyznaczony na drodze analizy empirycznej i na ogół przewyższa teoretyczny czasu wykonywania samego algorytmu. Zwykle jednak przyjmuje się założenie, że czas empiryczny programu $P$ jest proporcjonalny do czasu teoretycznego:

$$T_P = c \cdot T_A > T_A$$

dzięki czemu stosunki czasów otrzymanych dla różnych liczności danych są takie same dla algorytmu i programu:

$$\frac{T_A(n'')}{T_A(n')} = \frac{T_P(n'')}{T_P(n')} \quad n'' \neq n'$$

co pozwala porównać rezultaty analizy empirycznej i teoretycznej.

Zależność funkcyjna czasu $T(n)$ danego algorytmu może mieć dowolną postać i nieregularny przebieg, zwłaszcza w zakresie małych wartości $n$. Okazuje się, że dla większości algorytmów przebieg ten staje się regularny, gdy liczność $n$ przyjmuje duże wartości, co pozwala klasyfikować algorytmy w oparciu o asymptotyczne zachowanie ich czasów wykonywania. Algorytmy o podobnym zachowaniu asymptotycznym łączy się w jeden zbiór zwany klasą złożoności algorytmu.

➢ Mówimy, że algorytm A ma złożoność klasy O($f(n)$), gdy dla dowolnych dużych wartości $n$ zachodzi relacja:

$$T_A(n) \triangleq c \cdot f(n) \quad n >> 1$$

co oznacza, że czas wykonywania algorytmu jest asymptotycznie proporcjonalny (lub ograniczony z góry) do pewnej funkcji $f(n)$. Mówimy wobec tego, że czas ten jest rzędu O($f(n)$).

➢ Czasy działania algorytmów opisuje się przy pomocy następujących klas złożoności:

| | |
|---|---|
| O(1) | Większość instrukcji w programie wykonuje się raz lub najwyżej kilka razy. Jeżeli wszystkie instrukcje programu mają tę własność mówimy, że czas działania programu jest stały. |
| O(log n) | Jeżeli czas działania programu jest logarytmiczny to pooram nieznacznie spowalnia ze wzrostem n. Taka zależność występuję często w programach gdzie rozwiązywanie zadań wymaga transformacji na szereg zadań cząstkowych. |
| O(n) | Czas działania programu jest liniowy. Taką zależność mają algorytmy, które muszą przetwarzać n obiektów wejściowych lub generować n obiektów wyjściowych. |
| O(n log n) | Z taką zależnością mamy do czynienie, gdy algorytm rozbija problem na niezależne zadania cząstkowe, a następnie łączy otrzymane rozwiązania. |
| O($n^2$) | Gdy czas programu jest kwadratowy algorytm nadaje się tylko do rozwiązywania małych zadań. Ta zależność pojawia się wtedy, gdy program musi przejść przez pary elementów wejściowych. |
| O($n^3$) | Pojawia się wtedy, gdy algorytm przetwarza trójki elementów wejściowych |
| O($2^n$) | Bardzo mało jest takich algorytmów, gdy n się podwaja, czas rośnie do kwadratu. |
| O(n!) | Problemy NP- zupełne, dla których nie istnieje zależność wielomianowa, gdyż przetwarzają wszystkie permutacje n - elementowego ciągu danych wejściowych. |

➢ Wartości funkcji $f(n)$ dla typowych klas złożoności $O(f(n))$:

| n | lg n | sqrt(n) | n lg n | $n(\lg n)^2$ | $n^{3/2}$ | $n^2$ |
|---|---|---|---|---|---|---|
| 10 | 3 | 3 | 33 | 110 | 32 | 100 |
| 100 | 7 | 10 | 664 | 4410 | 1000 | 10000 |
| 1000 | 10 | 32 | 9966 | 99317 | 31623 | 1000000 |
| 10000 | 13 | 100 | 132877 | 1765633 | 1000000 | 10000000 |
| 100000 | 17 | 316 | 1660964 | 27588016 | 31622777 | 1000000000 |

➢ Czasy rozwiązywania zadań wielkiej skali w trybie obliczeń wysokiej wydajności dla podanego rozmiaru wejścia $n$ wykonywanych na maszynach o podanej szybkości obliczeń wyrażonej liczbą operacji na sekundę:

| Liczba operacji na sek. | Zadania o rozmiarze n ~ 1 milion | | | Zadanie o rozmiarze n ~ 1 miliard | | |
|---|---|---|---|---|---|---|
| | n | n lg n | $n^2$ | n | n lg n | $n^2$ |
| $10^6$ | Sekundy | Sekundy | Tygodnie | Godziny | Godziny | Nigdy |
| $10^9$ | Natychmiast | Natychmiast | Godziny | Sekundy | Sekundy | Dziesiątki lat |
| $10^{12}$ | Natychmiast | Natychmiast | Sekundy | Natychmiast | Natychmiast | Tygodnie |

# Obliczenia numeryczne w arytmetyce komputerowej

➢ Współczesny komputer cechuje się możliwością automatycznego, wielokrotnie powtarzalnego wykonywania dowolnego ciągu instrukcji, czyli kodu programu, którego celem jest przetwarzanie dowolnej informacji, czyli danych programu. Pod tym względem komputer jest przykładem uniwersalnej, programowalnej maszyny o ogólnym zastosowaniu – nie tylko liczącą.

➢ System komputerowy to bardzo złożony układ zawierający składniki fizyczne w postaci sprzętu komputerowego (elektronika) oraz składniki programowe (oprogramowanie). Sprzęt to przede wszystkim procesor CPU, pamięć i urządzenia peryferyjne.

➢ Komputery obecnie oferują również sprzętowe wsparcie dla obliczeń numerycznych. Każdy procesor zwiera bowiem obok podstawowej jednostki

logiczno-arytmetycznej ALU operującej bitowo na liczbach całkowitych także jednostkę FPU do przetwarzania liczb zmiennoprzecinkowych zwaną również jednostką przetwarzania numerycznego NPU lub potocznie koprocesorem matematycznym. Obecnie co raz częściej komputery wyposażone są w osobne karty obliczeniowe z procesorami graficznymi GPU i z własną jednostką FPU.

➡ Jednostka FPU realizuje właściwą arytmetykę komputerową, gdyż wykonuje podstawowe operacje numeryczne na liczbach rzeczywistych takie jak dodawanie, odejmowanie, mnożenie, dzielenie i obliczanie pierwiastka kwadratowego. Jednostka ta może również liczyć wartości różnych funkcji matematycznych takich jak logarytm, eksponent, funkcje trygonometryczne.

➡ Należy zauważyć, że jednostki FPU współczesnych komputerów oferują jedynie arytmetykę skończonej precyzji, co wynika z reprezentacji liczb w pamięci komputera. Arytmetyka dowolnej precyzji może być zatem dostępna jedynie przez użycie odpowiedniego oprogramowania najczęściej w postaci biblioteki numerycznej, której przykładem są LAPACK i BLAS.

➡ Liczba zmiennopozycyjna przetwarzana przez FPU jest reprezentowana w pamięci komputera w postaci bitów zwykle na dwóch (typ **half**), czterech (typ **single**), ośmiu (typ **double**) lub dziesięciu (typ **extended**) bajtach, a nawet 16 bajtach (typ **quad**), gdzie w nawiasach podano nazwy typów rzeczywistych, które mogą występować w językach programowania. Dokładność obliczeń numerycznych w arytmetyce komputerowej zależy w dużym stopniu od bajtowego rozmiaru reprezentacji liczb.

➡ Niezależnie od typu liczba zmiennopozycyjna jest zapisana w pamięci w formacie wykładniczym, a więc składa się z bitu ujemności (**sign**), bitów cechy (**exponent**) odpowiedzialnych za zakres wartości oraz bitów mantysy (**mantissa**) związanych z cyframi znaczącymi liczby. Na przykład liczba π reprezentowana z dokładnością do 24 bitów mantysy przy użyciu 8 bitowego wykładnika zawiera bity: **sign** = 0, **exponent** = 0000001, **mantissa** = 11001001000011111011011. Jak widać, przykład ten dotyczy 32-bitowego typu **single** dostępnego w języku C jako **float**, dla którego wartość liczby w dziesiętnym systemie pozycyjnym wynosi 3,141592, czyli z zaokrągleniem do jedynie 7 cyfr znaczących. Typ **double** obecny w C daje już 16 cyfr znaczących, zaś dostępny w PASCAL typ **extended** daje 20, natomiast **quad** dostępny w języku FORTRAN oferuje aż 33 cyfry znaczące.

Zapis bitowy liczby $x$ można w ogólności przedstawić w postaci wzoru:

$$x = s \bullet m \bullet b^{e-a}$$

gdzie $b = 2$ to podstawa zapisu binarnego w pamięci komputera, $s$ to bit ujemności równy 0 (plus) lub 1 (minus), $m$ to mantysa z przedziału $[1,b)$, czyli większa lub równa 1, stąd zapisywana bez wiodącej jedynki, zaś $e$ to cecha. Dodatkowy parametr $a$ to przesunięcie (**bias**) wykładnika, które dla typu **single** wynosi 127, a które pozwala uzyskać wykładniki ujemne. Tak otrzymana liczba w tym typie może się zatem zmieniać w zakresie $1.5 \cdot 10^{-45}$ .. $3.4 \cdot 10^{38}$. Dla typu **double** zakres ten wynosi $5.0 \cdot 10^{-324}$ .. $1.7 \cdot 10^{308}$.

W zapisie bitowym niektóre kombinacje bitów pełnią szczególną rolę, dzięki czemu możemy posługiwać sie liczbami specjalnymi **NaN** oraz ±**inf**. Pierwsza liczba powstaje, gdy procesor wykonał niedozwolona operację, druga zaś gdy zaszło dzielnie przez zero. Specjalny zapis posiada również wartość zerowa ±**null**.

Warto także zauważyć, że wszystkie prawa obowiązujące w arytmetyce matematycznej nie są spełnione w arytmetyce komputerowej. Mianowicie arytmetyka ta nie jest łączna, ale może być przemienna:

$$(x + y) + z \neq x + (y + z)$$

$$x + y = ? = y + x$$

co dotyczy również mnożenia liczb. W arytmetyce tej nie zachodzi również rozdzielność mnożenia względem dodawania:

$$x(y + z) \neq xy + xz$$

Ze względu na błędy zaokrągleń spowodowane skończoną reprezentacją liczb w pamięci równości te zachodzą jedynie z pewnym przybliżeniem. Innym przejawem tej reprezentacji jest istnienie niezerowej liczby $\varepsilon > 0$, dla której w arytmetyce komputerowej zachodzi dokładnie równość:

$$1 + \varepsilon = 1$$

Skutkiem skończonej reprezentacji liczb w pamięci jest również kasowanie się dokładności odejmowania dwóch liczb polegające na tym, że odjęcie dwóch bliskich liczb z małym błędem zaokrąglenia może w wyniku dać duży błąd dla ich różnicy. Zjawisko kasacji dokładności pojawia się szczególnie, gdy w komputerze wyznaczamy numeryczną pochodną funkcji na podatnie ilorazu różnicowego:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Zgodnie z zasadami analizy matematycznej przybliżenie to jest tym lepsze, im liczba $h$ jest bliższa zeru. Jednak przy użyciu operacji zmiennoprzecinkowych najmniejsza liczba nie daje najlepszego przybliżenia pochodnej. W miarę jak $h$ maleje różnica między $f(x+h)$ i $f(x)$ również maleje, kasując najbardziej znaczące i najmniej błędne cyfry zapisu i sprawiając, że najbardziej błędne cyfry nieistotne stają się ważne w tym zapisie. W rezultacie najmniejsza możliwa liczba $h$ daje bardziej błędne przybliżenie pochodnej niż nieco większa liczba. Jest to prawdopodobnie najpoważniejszy problem dokładności w arytmetyce komputerowej. Wobec tego numeryczne obliczanie pochodnej wprost na podstawie ilorazu różnicowego nie jest zbyt dokładną metodą. W schematach rozwiązywania równań różniczkowych stosuje się zamiast tego wyrażenia pozbawione różnic i ilorazów bliskich liczb.

## Obliczenia symboliczne w algebrze komputerowej

W matematyce i informatyce algebra komputerowa zwana również obliczeniami symbolicznymi lub obliczeniami algebraicznymi jest naukową dziedziną, której celem jest badanie i rozwój algorytmów i oprogramowania przeznaczonego do tekstowego manipulowania wyrażeniami matematycznymi.

Mimo iż algebrę komputerową można uznać za poddziedzinę obliczeń naukowych, to jest ona ogólnie uważane za odrębną dziedzinę, ponieważ obliczenia naukowe są zwykle oparte na metodach numerycznych z użyciem arytmetyki przybliżonych liczb zmiennoprzecinkowych, natomiast obliczenia symboliczne oparte są na dokładnych metodach z użyciem dowolnych wyrażeń zawierających symbole zmiennych. Symbole te nie mają żadnej wartości, stąd

obliczenia symboliczne opierają się na algorytmach przetwarzania tekstowego. Zwykle w tym celu używa się drzew składniowych i rekurencyjnej analizy zstępującej podobnie jak w przypadku translacji języków programowania.

Oprogramowanie wykonujące obliczenia symboliczne nazywane jest systemem algebry komputerowej. Użycie tu słowa system związane jest z ogromną złożonością tego oprogramowania. Na system algebry składa się bowiem metoda reprezentacji danych matematycznych w pamięci komputera jak również dedykowany menedżer pamięci, język skryptowy programowania w interfejsie użytkownika systemu, duży zestaw procedur do wykonywania typowych operacji, takich jak upraszczanie wyrażeń, różniczkowanie za pomocą reguły łańcuchowej, rozkład wielomianu na czynniki pierwsze, całkowanie nieoznaczone, obliczenie pochodnych, rozwiązywanie równań.

Najpopularniejszym systemem algebry komputerowej jest obecnie środowisko Wolfram System (WS). System ten jest przeznaczony do przetwarzania i ewaluacji skryptów obliczeniowych. Składa się co najmniej z dwóch programów, interfejsu GUI w postaci interakcyjnego edytora notatnikowego o nazwie Mathematica.exe (tzw. front-end) oraz kernela obliczeniowego o nazwie WolframKernel.exe lub o innej nazwie MathKernel.exe (tzw. back-end). Programy te są wykonywane w ramach oddzielnych procesów systemu operacyjnego wymieniających informację symboliczną poprzez komunikację międzyprocesową z użyciem specjalnego protokołu Wolfram Symbolic Transfer Protocol (WSTP).

W oknie edytora jest wprowadzany i wyświetlany tekst w formie interaktywnego notatnika. Tekst skryptowy zapisany jest w języku Wolfram Language (WL). Jest to język o dynamicznej typizacji danych oraz o funkcyjnym paradygmacie programowania. W gramatyce tego języka obok znaków operatorów i separatorów występują tzw. atomy, czyli główne jednostki leksykalne i niektóre konstrukty składniowe, które jako ciągi znaków alfabetu języka reprezentują w skrypcie dane. Gramatyka funkcyjna zawiera jednakże przede wszystkim funkcje, które wykonują operacje na atomach, oraz transformacje operujące na funkcjach i atomach, czyli obiektach programu. Zarówno funkcje jak i transformacje pełnią rolę konstrukcji składniowych języka. Z punktu widzenia paradygmatu funkcyjnego elementy te powinny reprezentować czysty, transparentny kod, podobnie jak atomy powinny w tym paradygmacie reprezentować czyste, niemutacyjne dane.

Po wprowadzeniu skryptu, wyświetlanego w formie tekstu wejściowego, edytor wraz z kernelem wykonuje jego translację, która poprzedzona jest wstępną konwersję tekstu, a następnie przebiega przez właściwą interpretację językową, redukującą skrypt źródłowy do postaci wynikowej, co nazywamy również ewaluacją (wartościowaniem) skryptu. W ewaluacji tej uczestniczy również kernel, który wykonuje główne obliczenia opisane skryptem. Wyniki ewaluacji skryptu źródłowego są wyprowadzane do okna edytora i wyświetlane w formie tekstu wyjściowego, co może być poprzedzone końcową konwersją tekstu. Teksty wejściowe i wyjściowe mogą być przy tym zastąpione formą graficzną w edytorze z graficznym interfejsem użytkownika. Teksty te co więcej mogą również odnosić się do nietekstowych obiektów systemu, np. obrazów i dźwięków zapisanych w plikach binarnych.

Niektóre czynności w procesie przetwarzania skryptów można w pewnym stopniu jawnie wykonać. Na przykład tekst wejściowy jako skrypt źródłowy można jawnie zamienić na pojedyncze wyrażenie języka przy użyciu operacji ToExpression. Otrzymane wyrażenie poddajemy następnie dalszej translacji (ewaluacji) do postaci wynikowej. Natomiast końcowe wyrażenie języka otrzymane w rezultacie tego procesu można jawnie przekształcić na tekst wyjściowy (skrypt wynikowy) przy użyciu operacji ToString.

Notatnik Notebook edytora podzielony jest w pionie na poziome komórki Cell o danym stylu. Każda komórka jak i cały notatnik posiadają własne formatowanie, co można ustawić w głównym menu Format i Cell, a także w menu lokalnym z prawej strony komórki. Komórki są zorganizowane zewnętrznie w postaci grup komórek CellGroup, a ich treść wewnętrznie osadzona jest w niewidocznych pudełkach Box stanowiących cześć niskopoziomowego języka znaczników w notatniku. W komórce wejściowej o stylu Input zamieszczony jest tekst edycji skryptu, natomiast tekst ewaluacji skryptu znajduje się w komórce wyjściowej o stylu Output.

W trakcie procesu przetwarzania skryptu wyniki obliczeń w postaci dowolnego wyrażenia języka Wolfram mogą być drukowane wprost do ogólnego strumienia o nazwie $Output przy użyciu operacji Print. Strumień ten w szczególności może być skojarzony z osobnym oknem konsoli wyświetlanym poleceniem Messages z menu głównego Window. Jeżeli jednak jest skojarzony z oknem głównym edytora, wtedy wydruk pojawia się w przestrzeni pomiędzy komórkami we/wy. Należy zauważyć, że rezultatem ewaluacji operacji wydruku Print jest symbol pustki lub też zera Null, który

zwykle nie jest wyświetlany w komórce wyjściowej. W związku z tym wydruk do strumienia wyjściowego należy w programie funkcyjnym traktować jako efekt uboczny. Inne operacje w systemie Wolfram dotyczące wydruku (wyświetlania) wyników nie traktują tej czynności jako efekt uboczny. Są to zatem operatory funkcyjne, które w swoim rezultacie jedynie dodają odpowiedni nowy korzeń do drzewa składniowego podanego wyrażenia języka, tak aby jego wynikowa postać tekstowa (graficzna) w komórce wyjściowej została odpowiednio zmodyfikowana. Przykładem takich operatorów są operacje FullForm, BaseForm, TreeForm, które możemy również traktować jako dyrektywy wydruku.

- W systemie Wolfram istnieje ponadto możliwość wykonywania odwrotnej czynności niefunkcyjnej, czyli skanowania danych przeznaczonych dla obliczeń, co w trybie tekstowym odbywa się ze strumienia wejściowego $Input, natomiast w trybie interakcyjnym odbywa się w osobnym oknie graficznym. Czynność tę wykonuje operacja monadyczna Input, a jej rezultatem jest odczytane wyrażenie języka, które zwykle przekazujemy do dalszej translacji w systemie.

- W notatniku edytora skrypt można poddać osobnej ewaluacji w każdej komórce wejściową poleceniem Evaluate Cell z menu kontekstowego lub skrótem Shift-Enter, ale można również ewaluować cały skrypt we wszystkich komórkach wejściowych notatnika poleceniem Evaluate Notebook z menu głównego Evaluation. Komórki o stylu innym niż Input (np. Text) zwykle nie są poddawane ewaluacji. Po każdej ewaluacji powstają nowe komórki o stylu Output zawierające wyniki ewaluacji w formie tekstowej lub graficznej. Komórki te można każdorazowo usunąć przed następną ewaluacją skryptu poleceniem Delete All Output z menu głównego Cell. W ogólności zatem przetwarzanie informacji w środowisku WS z udziałem komórek Input / Output odbywa się w ramach głównej pętli REPL, czyli READ-EVALUATE-PRINT LOOP, podobnie jak to zachodzi w środowisku Python. Środowisko w trybie interakcyjnym każdorazowo czeka na wprowadzenie danych do komórki Input, a następnie na żądanie użytkownika czyta dane i wykonuje ich ewalaucję, po czym drukuje wyniki do komórki Output.

- Treść notatnika wyświetlana w oknie edytora w formie tekstu i grafiki znajduje się zarówno w pamięci edytora, jak i w jego pliku (*.nb) na dysku. Treść ta wewnętrznie zapisana jest przy użyciu osobnego języka znaczników, który wbudowany jest do głównego języka WL i zawiera niskopoziomowe

polecenia dot. struktury notatnika, formatowania tekstu oraz stylu jego wyświetlania w oknie edytora. Polecenia te są wprost interpretowane przez edytor, kernel zaś może je używać jedynie, aby wyświetlić treść z wynikami obliczeń. Polecenia edytora można również jawnie użyć w samym skrypcie, aby zarządzać strukturą, formą i stylem jego notatnika.

➡ Plik notatnika oraz jego treść w edytorze mogą być przetwarzane poprzez standardowe polecenia z menu głównego File, Edit, Insert, Window. Między innymi w menu File poleceniem Save As można wydrukować treść notatnika do formatu PDF, co w szerszym zakresie oferuje również polecenie Print. Treść notatnika w formie graficznej można konfigurować w menu głównym Graphics. Natomiast polecenia menu głównego Palettes pozwalają wprowadzać do tekstu notatnika znaki specjalne (np. Special Characters) oraz matematyczną notację dla danych wejściowych (np. Basic Math Assistant).

➡ Zamiast graficznego edytora notatnikowego istnieje w środowisku WS możliwość wsadowego użycia programu do interpretacji skryptów WL. W tym celu program interpretera o nazwie wolframscript.exe należy uruchomić w linii poleceń konsoli z opcją -code, za którą podajemy tekst skryptu, lub z opcją -file, za którą podajemy nazwę zwykłego pliku tekstowego (*.wl) zawierającego skrypt. Program ten może być również uruchomiony w konsoli bez opcji, a wtedy oferuje funkcjonalność interakcyjnego edytora skryptów z tekstowym interfejsem użytkownika bez możliwości wyświetlania grafiki. Dla obydwu trybów pracy interpretera skryptowego, wsadowego i interakcyjnego silnik (krenel) obliczeniowy Wolfram Engine to program o nazwie wolfram.exe lub o innej nazwie math.exe.

## Prezentacja komputerowa wyników obliczeń

➡ Współczesne systemy operacyjne komputerów z graficznym interfejsem użytkownika GUI oferują bogatą prezentację komputerową wyników obliczeń. Zwykle wyniki są prezentowane w postaci wykresów graficznych dwu- lub trójwymiarowych na podstawie tabelek z danymi i wynikami. Co raz częściej jednak wyniki obliczeń prezentowe są w formie komputerowej grafiki lub animacji 3D, co na przykład umożliwia pełną wizualizację stanu układu badanego w symulacji komputerowej.

➡ W ogólności przetwarzanie obrazu graficznego posiada ogromne znacznie dla komunikacji człowieka z komputerem. Obecnie bowiem zarówno

koncepcje teoretyczne dotyczące obrazu, jak i graficzne urządzenie wejścia/wyjścia stanowią właśnie podstawę dla tworzenia interfejsów GUI w systemach komputerowych.

W technicznych środkach informacji zagadnienia związane z obrazem jako dwuwymiarowym zbiorem punktów dzielimy na trzy grupy:

- rozpoznawanie obrazu - istniejący obraz wejściowy (najczęściej analogowy zamieniony na postać cyfrową) jest analizowany przez odpowiednie programy w celu wyodrębnienia istotnych informacji w nim zawartych, zapisywanych na wyjściu w formie opisu tekstowego;
- przetwarzanie obrazu - istniejący obraz wejściowy przekształcany jest na obraz wyjściowy, przy czym zmiana może dotyczyć np. formatu obrazu, jego rozdzielczości lub może polegać na zwiększeniu czytelności obrazu;
- synteza obrazu - nowy obraz wyjściowy jest tworzony metodami cyfrowymi na podstawie wejściowego opisu tekstowego (modelu).

Grafika komputerowa zajmująca się syntezą obrazu obejmuje zagadnienia matematycznego opisu aspektów geometrycznych i fizycznych tworzonego obrazu, zagadnienia związane z projektowaniem algorytmów syntezy obrazu i odpowiedniego oprogramowania, a także zajmuje się sprzętem graficznym, w którym utworzony obraz jest najczęściej reprezentowany w postaci dwuwymiarowej matrycy barwnych punktów, czyli tzw. pikseli.

Obecnie grafika komputerowa postrzegana jest nie tylko jako zbiór technicznych metod tworzenia obrazów komputerowych, ale przede wszystkim jako rozległa dziedzina o charakterze interdyscyplinarnym zajmująca się wizualizacją złożonych układów wirtualnych. Tworzeniem opisu takich układów zajmuje się modelowanie komputerowe oparte nierzadko na prawach fizyki i metodach numerycznych. Układ wirtualny jest najczęściej komputerowym modelem pewnego fragmentu rzeczywistości. Dzięki metodom grafiki komputerowej można nie tylko badać jakościowo model i odpowiadający mu fragment rzeczywistości, ale również prezentować wyniki obliczeń w formie realistycznych obrazów i animacji.

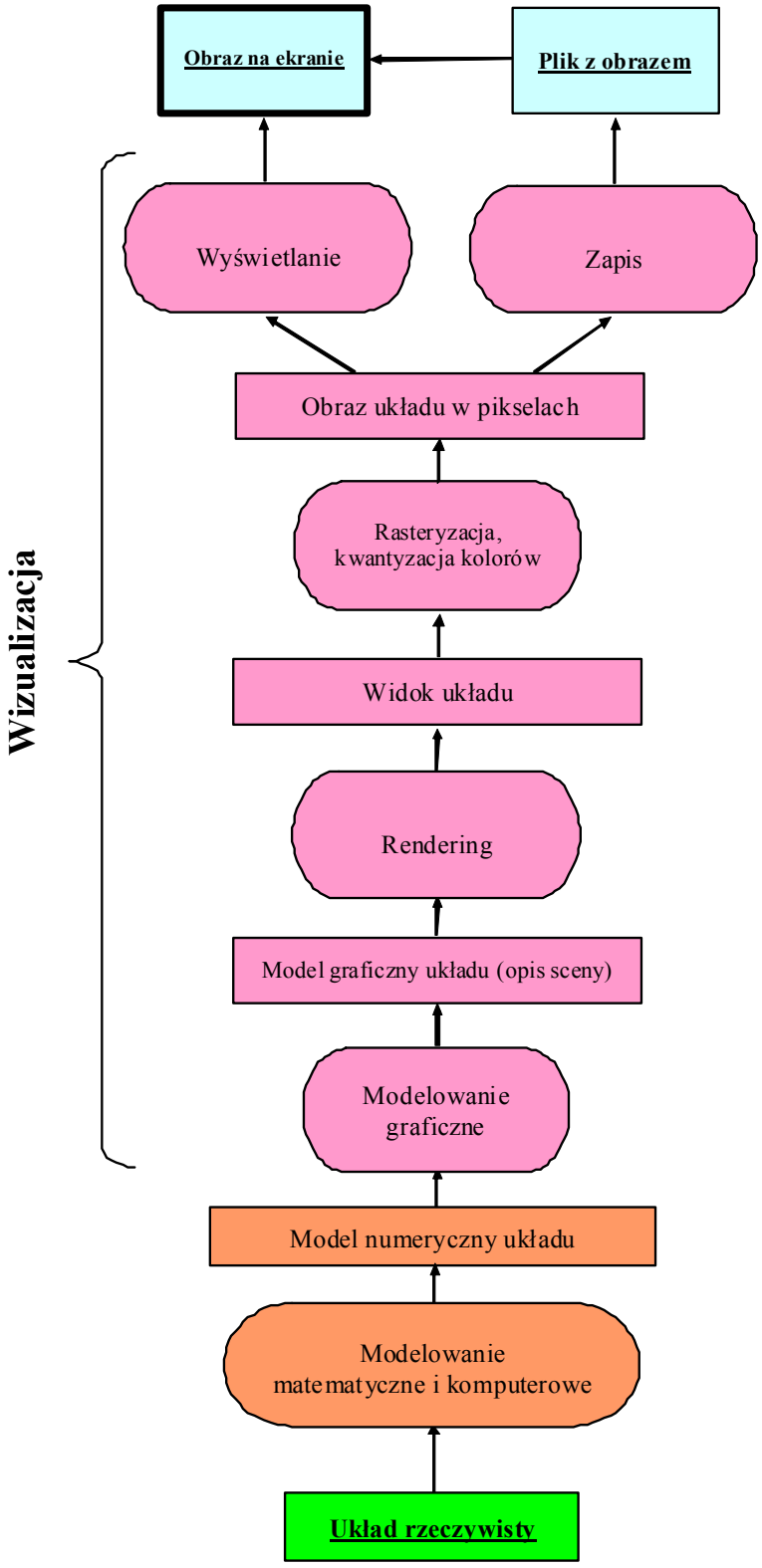
Obszar zastosowań grafiki komputerowej wykracza zatem znacznie poza tradycyjne metody rysowania obrazów dla potrzeb artystycznych, czy inżynierskich. Z metod tej dziedziny korzysta się powszechnie do prezentacji wyników złożonych obliczeń i eksperymentów naukowych, w symulacji komputerowej rozmaitych układów, zjawisk, procesów fizycznych dla potrzeb nauki i techniki, w dydaktyce wspomaganej komputerowo, w przemyśle rozrywkowym np. do tworzenia efektów specjalnych w filmach, filmów animowanych i gier komputerowych, a także w systemach symulacyjnych czasu rzeczywistego, takich jak symulatory lotu.

Z punktu widzenia grafiki komputerowej wejściowy układ wirtualny podlegający wizualizacji nazywamy sceną. Scena może być modelem pewnego fragmentu rzeczywistości lub też może mieć dowolnie abstrakcyjny charakter. Opis sceny tworzony w procesie modelowania zawiera przede wszystkim parametry geometryczne i fizyczne obiektów wchodzących w skład sceny.

Informacja zawarta w opisie sceny zależy od wymiarowości sceny. W przypadku układów dwuwymiarowych do pełnej wizualizacji potrzeba jedynie parametrów geometrycznych, takich jak położenie, rozmiary, kształt obiektów, oraz określenia kolorów i tekstury (wzoru wypełnienia) obiektów. Podanie jedynie takiej informacji w odniesieniu do scen trójwymiarowych powoduje, że otrzymany obraz nie posiada wrażenia głębi. Dopiero określenie modelu i parametrów oświetlenia wprowadza poprzez grę świateł i cieni wrażenie głębi do obrazu obserwowanego przez człowieka. Ponieważ wyjściowy obraz jest zawsze płaskim zbiorem punktów, więc w opisie sceny trójwymiarowej istotne jest również określenie sposobu zrzutowania sceny na płaszczyznę obrazu. Synteza obrazów scen trójwymiarowych jest zatem znacznie bardziej skomplikowana niż w przypadku scen dwuwymiarowych.

Tworzenie realistycznych obrazów scen 3D jest procesem wielo-etapowym o charakterze potokowym, co przedstawia diagram na następnej stronie. Na proces ten składają się następujące trzy główne etapy:

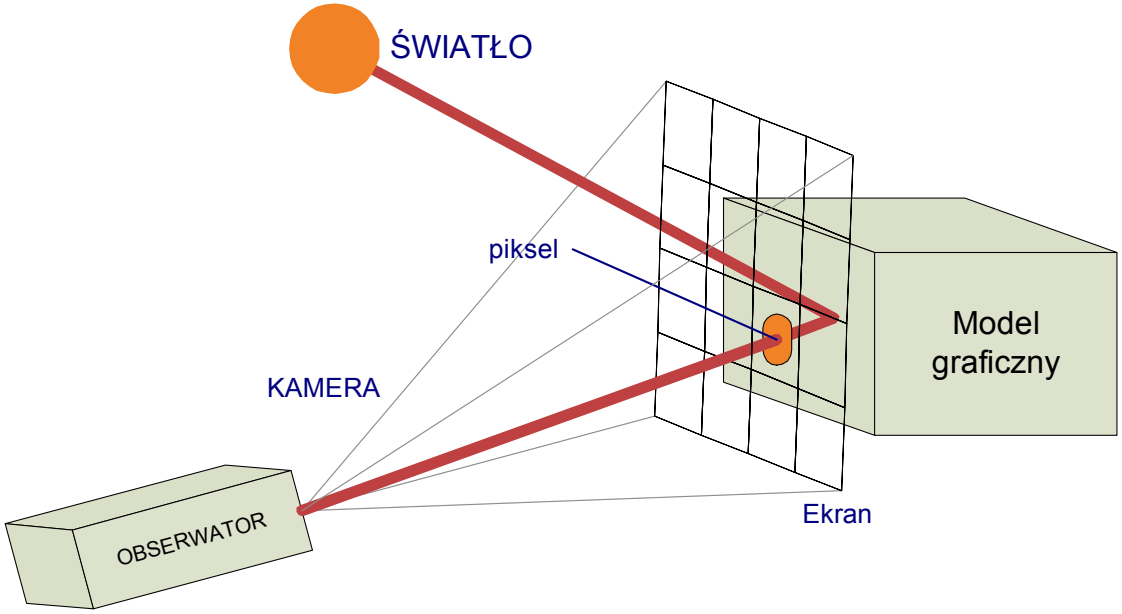
- modelowanie
- rendering
- wyświetlenie (zapis) z rasteryzacją i kwantyzacją




Pierwszy etap dotyczy tworzenia modelu świata, czyli geometrycznego i fizycznego opisu sceny, dwa następne etapy to właściwa wizualizacja sceny. Modelowanie komputerowe może być przeprowadzane zupełnie niezależnie od

wizualizacji jako składowa pewnego procesu obliczeniowego (np. symulacji). Utworzony w wyniku tego procesu układ wirtualny jest zwykle w opisie sceny zastępowany prostszym modelem graficznym. Opis sceny może być również tworzony przy pomocy specjalistycznych programów narzędziowych przeznaczonych do modelowania obiektów, postaci dla potrzeb filmów, gier.

Obiekty opisane w scenie są zbudowane z figur elementarnych, tzw. prymitywów, którymi mogą być linie, trójkąty lub inne wielokąty powstałe poprzez połączenie wierzchołków, czyli tzw. werteksów. Położenie werteksów obiektu podawane jest w układzie trzech współrzędnych (x,y,z). Jest to najczęściej układ lokalnych współrzędnych, względem którego określony jest dany obiekt. Dla każdego werteksu obiektu w scenie należy podać jego kolor, a także pewne właściwości fizyczne materiału (tekstury) pokrywającego dany obiekt. Należy również w werteksie określić kierunek wektora normalnego do powierzchni obiektu. Opis sceny zawiera również charakterystykę jej oświetlenia, a także właściwości ośrodka wypełniającego scenę. W celu dopełnienia opisu sceny należy podać również położenie obserwatora sceny względem jej obiektów. Przedstawia to poglądowo poniższy rysunek.




Utworzony opis sceny jest następnie przekazywany na wejście do kolejnego etapu, renderingu. Na etap ten składają się przede wszystkim przekształcenia układu współrzędnych, względem którego określone są obiekty sceny. Przekształcenia przeprowadzane są od lokalnych współrzędnych modelu obiektu, do globalnych współrzędnych sceny. Następnie współrzędne sceny zamieniane są na współrzędne obserwatora sceny związanego z ekranem, na

którym powstanie docelowy obraz. Ze względu na ograniczony kąt widzenia obserwatora niektóre obiekty są obcięte przy przejściu do współrzędnych obserwatora. Czasami zamiast pojęcia obserwatora wprowadza się pojęcie kamery filmującej scenę. Do tego momentu obiekty opisywane są trzema współrzędnymi. Kolejny krok to przekształcenie tych współrzędnych na współrzędne płaszczyzny obrazu, czyli zrzutowanie obiektów na pewną abstrakcyjną stronę o znanych rozmiarach. Współrzędne określone na płaszczyźnie strony mogą być podane w fizycznych jednostkach, takich jak milimetry, cale. W ten sposób na płaszczyźnie strony powstaje obraz sceny składający się z punktów o odpowiednim kolorze. Punkty te oprócz współrzędnych strony zachowują dodatkowo wartość współrzędnej 'z', czyli odległość punktu od płaszczyzny strony; informacja ta zostanie wykorzystana dalej do badania przysłaniania się obiektów w scenie i usunięcia powierzchni niewidocznych. W procesie rzutowania należy ponadto uwzględnić kolory i normalne werteksów, właściwości materiałowe, a także charakterystykę oświetlania i ośrodka sceny. Wszystkie te aspekty wpływają na kolor punktów w docelowym obrazie. Oświetlenie obiektów sceny można badać w trakcie renderingu posługując się algorytmami opartymi na bardziej fizycznych podstawach. Obecnie używa się dwóch takich algorytmów: metody śledzenia promieni światła (ang. *ray tracing*) oraz metody energetycznej opartej na prawie Lamberta.

Ostatnim etapem tworzenia obrazu jest wyświetlenie go na wybranym urządzeniu wyjściowym, takim jak karta grafiki wraz z monitorem, drukarka, ploter lub zapis do pliku graficznego na dysku. W tym celu obraz otrzymany z renderingu poddany jest przekształceniu do współrzędnych urządzenia (np. ekranu) mierzonych w pikselach, czyli najmniejszych elementach obrazu (ang. *pixel* = *picture element*). W ten sposób każdy punkt obrazu otrzymanego z renedringu może być zastąpiony przez jeden lub kilka pikseli zgodnie z bieżącą rozdzielczością wybranego urządzenia, czyli ilością pikseli na jeden cal (może też zajść przypadek odwrotny, gdy kilka punktów obrazu zastępowane jest przez jeden piksel - jest to przypadek niekorzystny dla jakości wyświetlanego obrazu). Proces przekształcania punktów obrazu na piksele zwany jest również rasteryzacją obrazu. W trakcie rasteryzacji może być wykonane ostateczne badanie przysłaniania obiektów w scenie i usunięcie niewidocznych powierzchni. Wykorzystuje się w tym celu sortowanie prymitywów ze względu na odległość od ekranu lub metodę bufora głębi (z-bufora). Ponadto, w trakcie rasteryzacji wykonywane jest wypełnianie kolorem pustych obszarów

pikselowych powstałych z połączenie werteksów sceny; można stosować przy tym metodę cieniowania poprzez interpolację kolorów. W rezultacie tego procesu powstaje końcowy obraz składający się z barwnych pikseli. Kolor pikseli odpowiadających werteksom sceny ustalany jest również w trakcie rasteryzacji i zależy od koloru punktów wejściowego obrazu, a także od palety kolorów jednocześnie wyświetlanych w wybranym urządzeniu. Zwykle ilość tych kolorów jest znacznie mniejsza w porównaniu do palety kolorów otrzymanych po renderingu, a zatem wyświetlenie obrazu wymaga ograniczenia palety jego kolorów. Proces ten nazywamy kwantyzacją koloru.

## Zintegrowane środowiska obliczeniowe

- Współczesne naukowe programy komputerowe oferują zintegrowane środowisko obliczeniowe, które umożliwia wykonywanie głównie obliczeń numerycznych, ale istnieje również możliwość wykonywania obliczeń symbolicznych. Przykładem takiego środowiska jest przedstawiony powyżej system Wolfram. Przede wszystkim jednak środowiska te udostępniają bogaty interfejs użytkownika oparty na graficznym trybie wyświetlania z pełną prezentacją komputerową wyników obliczeń. Wprowadzanie danych do obliczeń odbywa się przy tym w oknach dialogowych charakterystycznych dla graficznego interfejsu użytkownika, jak również poprzez notatnikowy interfejs z użyciem własnego języka skryptowego.

- Przykładami takich programów są Excel, MATLAB, Maple, a także pakiety języka Python takie jak Scilab, Anaconda, SageMath. Dostępne są również programy do obliczeń inżynierskich takie jak Ansys, Fluent, Flux.

- Ansys INC to przedsiębiorstwo, które zajmuje się produkcją i dystrybucją oprogramowania do wspomagania sterowania procesami technologicznymi. Jest wiele rodzajów programów oferowanych nam przez Ansy INC. Do najbardziej rozbudowanych zaliczyć należy: *Multiphysics, Mechanical*. Wspomnieć należy, iż *Ansys* jest wiodącym na świecie pakietem oprogramowania, w którym obliczenia przeprowadzane są za pomocą metody MES. Umożliwia to kompleksową symulację w każdej dziedzinie nauki i przemysłu. Mimo, iż pakiety firmy Ansys nie należą do najłatwiejszych w obsłudze to komfortowy interfejs graficzny umożliwia po krótkim wprowadzeniu niedoświadczonemu użytkownikowi dokonywanie pierwszych

analiz. Ansys składa się z wielu narzędzi zaimplementowanych w pakiecie. Zastosowanie metody elementów skończonych powoduje duże korzyści. Możliwe staje się projektowanie optymalnych pod wieloma względami konstrukcji (np. o minimalnym ciężarze, energooszczędnych itd.) drastycznie maleje liczba kosztownych prototypów, skrócony jest znacznie czas wprowadzania produktu na rynek.

Następnym programem przeznaczonym do obliczeń inżynierskich jest *Fluent.* Program ten rozwiązuje wszelkie problemy, które są związane z jakimkolwiek przepływem substancji ciekłej. Obliczenia, które wykonuje się za pomocą pakietu Fluent bazują na metodzie skończonych objętości. Dzięki temu możliwe jest wykorzystanie nie tylko niestrukturalnej siatki obliczeniowej, ale możliwe jest także przeprowadzenie obliczeń w parciu o siatki o strukturze blokowej. W przeciwieństwie do typowych programów z zakresu dynamiki cieczy możliwa jest tutaj duża swoboda w wyborze modelu fizycznego, algorytmu numerycznego czy też schematów dyskretyzacji pochodnych przestrzennych i czasowych. Ta zaleta programu Fluent niestety jest źródłem błędów, generowanych przez program obsługiwany przez niedoświadczonego użytkownika. Kolejną zaletą Fluent'a to możliwość definiowania własnych modułów tzw. *user defined function*, odpowiedzialnych za odwzorowanie własności materiałów, a także za modyfikacje algorytmu numerycznego. Fluent to przede wszystkim program, który rozwiązuje problemy z przepływem substancji. Dostępne są wszystkie rodzaje przepływów: od niskiego ciśnienia do przepływów ponaddźwiękowych. Program posiada wysoką możliwość konfiguracji co prowadzi do optymalnego dobrania dokładności dla szerokiej gamy przepływów. Fluent posiada bogatą bibliotekę różnych modeli, co daje nam możliwość właściwego wyboru rodzaju przepływu, np.: laminarnego, przejściowego, turbulentnego. Dostępne są także różnego rodzaju modele przepływu ciepła czy przepływy wielofazowe itp. Program uwzględnia strukturę i rodzaj roztworu badanej substancji.

Flux 3d to program oparty o metodę elementów skończonych (MES), który służy do wykonywania elektromagnetycznych i termicznych symulacji w wymiarze 3d jak i 2d. Jest to podstawowy program do projektowania i analizowania wszystkich urządzeń elektromagnetycznych. Dzięki unikalnym możliwościom tj.: prowadzenie wieloparametrycznych obliczeń, realizacji zaawansowanych symulacji polowych sprzężonych z zewnętrznymi obwodami elektrycznymi czy sprzężonymi kinetycznymi program ten jest doskonałym

narzędziem dla harmonicznych, przejściowych a także statycznych analiz. Dodatkowo możemy tworzyć komendy oparte o język Pythona, tworzyć nowy program wykonawczy poprzez dołączenie własnych procedur w języku Fortran. W przeciwieństwie do poprzednio omawianych programów Flux nie jest otwartym pakietem osobnych programów (modułów), zamiast tego stanowi środowisko obliczeniowe o zamkniętej architekturze.